```
MFCC_from_Mel.py

import numpy as np

def dct_type2_orthogonal(matrix, n_mfcc=20):
    """
    Computes the Type-II DCT along the rows of a matrix with orthogonal normalization.

    matrix: Log-mel spectrogram matrix of shape (n_mels, num_frames)
    n_mfcc: Number of cepstral coefficients to retain (usually 13 to 20)
    """
    n_mels, num_frames = matrix.shape
    mfcc = np.zeros((n_mfcc, num_frames))

    # Precompute indices for vectorization over frames
    m_indices = np.arange(n_mels)

    for c in range(n_mfcc):
        # Scale factor for orthogonal normalization
        if c == 0:
            ortho_scale = np.sqrt(1 / n_mels)
        else:
            ortho_scale = np.sqrt(2 / n_mels)

        # Compute the DCT equation for coefficient 'c' across all time frames
        # Cosine term: cos( pi * c * (2m + 1) / (2 * n_mels) )
        cos_term = np.cos(np.pi * c * (2 * m_indices + 1) / (2 * n_mels))

        # Multiply along the Mel-axis and scale
        mfcc[c, :] = ortho_scale * np.dot(cos_term, matrix)

    return mfcc

def compute_mfcc_from_scratch(signal, sr, n_mfcc=13, **kwargs):
    """
    Complete pipeline to extract MFCCs directly from raw audio.
    """
    # 1. Get the Log-Mel Spectrogram (using your previous function)
    log_mel_spec = mel_spectrogram_from_scratch(signal, sr, **kwargs)

    # 2. Apply the Orthogonal Type-II DCT and truncate to n_mfcc
    mfccs = dct_type2_orthogonal(log_mel_spec, n_mfcc=n_mfcc)

    return mfccs


# Generate the custom MFCCs (keeping the standard 13 coefficients)
mfccs = compute_mfcc_from_scratch(signal, sr=sr, n_mfcc=13)

# Display the result
plt.figure(figsize=(10, 4))
plt.imshow(mfccs, aspect='auto', origin='lower', cmap='coolwarm')
plt.title('Custom MFCCs From Scratch')
plt.ylabel('MFCC Coefficient Index')
plt.xlabel('Time Frame Index')
plt.colorbar(label='Amplitude')
plt.tight_layout()
plt.show()


```